

lks_demo 扫描画布结构说明

目的：说明`lks\_demo`画布由 LKS 源码扫描生成后的层级结构、各模块功能、内部公式和后续可读性优化建议。

1. 结论

`lks\_demo` 不是手工搭建的旧版 LKS 画布，而是 GAASD 根据 LKS C++ 源码扫描生成的 Pangu 模块画布。

当前画布能够和源码结构对应上：

画布层级	画布名称	对应源码类/函数	作用
应用层	空应用`app_empty`	GAASD 工程容器	承载模块
模块层	LKS参考模块`LKSMModule`	Pangu`LKSMModule`	对接`lks_input` / `lks_output` 通道
算法入口	LKS Main Flow`	`control::LKSAAlgorithm::run()`	调度 LKS 四个子功能并汇总输出
子模块	`Enable Logic`	`DetermineLksControlEnabled::run()`	判断 LKS 是否允许输出控制
子模块	`Preview Distance`	`ComputeLksPreviewDistance::run()`	根据车速和曲率计算远预瞄距离
子模块	`Weighted Lane Error`	`ComputeLksWeightedError::run()`	计算近/中/远三点横向误差并加权
子模块	`Steer Command`	`ComputeLksSteerCommand::run()`	根据加权误差输出限幅后的转向命令

源码缓存位置：

```
/home/aiden/gaasd_versions/gaasd-2.7.0.5/home/gaasd_server/components/THICV/extracted_c_functions/final_codescan/sources/lks_input_controller_3587345e
```

画布数据库位置：

```
/home/aiden/文档/Modularization/lks_demo/data/cbdes.db
```

2. 顶层模块结构

2.1 `LKSMModule`

`LKSMModule` 是 Pangu 节点模块层，包含：

子项	类型	作用
`lks_input`	输入通道	订阅 LKS 所需输入消息
`lks_output`	输出通道	发布 LKS 计算结果
`run`	复合算法块	执行车道保持算法

2.2 输入通道`lks\_input`

字段	类型	含义
`ego_speed_mps`	`double`	自车速度, 单位 m/s
`c0_m`	`double`	车道线三次多项式常数项, 单位 m
`c1`	`double`	车道线一次项
`c2_per_m`	`double`	车道线二次项, 单位 1/m
`c3_per_m2`	`double`	车道线三次项, 单位 1/m ²
`curvature_per_m`	`double`	道路曲率, 单位 1/m
`brake_pressed`	`bool`	制动踏板状态
`driver_steer_norm`	`double`	驾驶员方向盘接管输入, 归一化
`frame_id`	`uint32`	帧编号, 透传到输出

2.3 输出通道 `lks\_output`

字段	类型	含义
`frame_id`	`uint32`	输入帧编号透传
`steer_rad`	`double`	LKS 输出方向盘转角命令
`control_enabled`	`bool`	LKS 控制是否使能
`valid`	`bool`	输出是否有效
`preview_distance_m`	`double`	当前远预瞄距离
`weighted_error_m`	`double`	近/中/远三点加权横向误差

3. 算法入口 `LKSAgorithm::run`

画布名称：`LKS Main Flow`

源码功能：

```
void LKSAgorithm::run(const Input& input, Output& output)
```

输入：

```
egoV, curvature, c0, c1, c2, c3, brakePressed, driverSteerNorm
```

输出：

```
lksSteerRad, controlEnabled, valid, previewDistance, weightedError
```

执行顺序：

1. 调用 `DetermineLksControlEnabled` 得到 `controlEnabled`。
2. 调用 `ComputeLksPreviewDistance` 得到 `previewDistance`。
3. 调用 `ComputeLksWeightedError` 得到 `weightedError`。
4. 调用 `ComputeLksSteerCommand` 得到 `lksSteerRad`。
5. 输出 `valid = 1.0`，并把中间观测量同步输出。

整体数据流：

```
lks_input
-> LKSAlgorithm
  -> DetermineLksControlEnabled
  -> ComputeLksPreviewDistance
  -> ComputeLksWeightedError
  -> ComputeLksSteerCommand
-> lks_output
```

4. 子模块一：LKS 控制使能判断

画布名称：`Enable Logic`

源码类：

```
DetermineLksControlEnabled
```

输入：

输入	含义
`egoV`	自车速度
`brakePressed`	制动是否触发
`driverSteerNorm`	驾驶员方向盘输入

参数：

参数	当前值	含义
`vMin`	`1.0`	LKS 允许工作的最低车速
`driverSteerThreshold`	`0.1`	驾驶员方向盘接管阈值

内部逻辑：

```
speedReady = egoV >= vMin
brakePressed = brakePressed != 0
absDriverSteerNorm = abs(driverSteerNorm)
driverOverride = absDriverSteerNorm >= driverSteerThreshold

controlEnabled = speedReady AND (NOT brakePressed) AND (NOT driverOverride)
```

作用：

车辆速度过低、驾驶员踩刹车、驾驶员主动打方向时，LKS 控制输出被关闭。

5. 子模块二：LKS 远预瞄距离计算

画布名称：`Preview Distance`

源码类：

ComputeLksPreviewDistance

输入：

输入	含义
----- ----	
`egoV`	自车速度
`curvature`	道路曲率

参数：

参数	当前值	含义
----- ----- -----		
`l0`	`5.0`	基础预瞄距离
`rt`	`0.5`	速度相关预瞄系数
`curvatureThreshold`	`0.001`	曲率阈值
`rAlpha`	`0.6666667`	弯道预瞄缩放系数

内部逻辑：

```
basePreviewDistance = l0 + rt * egoV
absCurvature = abs(curvature)

if absCurvature >= curvatureThreshold:
    curvatureScale = rAlpha
else:
    curvatureScale = 1.0

previewDistance = basePreviewDistance * curvatureScale
```

作用：

直线时预瞄距离随车速增大；弯道时按`rAlpha`缩短预瞄距离，提高弯道响应速度。

6. 子模块三：LKS 三点预瞄误差加权

画布名称：`Weighted Lane Error`

源码类：

ComputeLksWeightedError

输入：

输入	含义
----- -----	
`c0`	三次车道多项式常数项
`c1`	一次项
`c2`	二次项
`c3`	三次项
`previewDistance`	远预瞄距离

参数：

参数	当前值	含义	
nearPreviewDistance	0.5	近预瞄点 x 坐标	
w1	0.2	近点误差权重	
w2	0.3	中点误差权重	
w3	0.5	远点误差权重	

三个预瞄点：

```
nearX = nearPreviewDistance
middleX = previewDistance / 2
farX = previewDistance
```

车道线多项式：

$$e(x) = c3 \cdot x^3 + c2 \cdot x^2 + c1 \cdot x + c0$$

源码中使用 Horner 形式实现：

$$e(x) = (((c3 \cdot x) + c2) \cdot x + c1) \cdot x + c0$$

加权误差：

```
nearError = e(nearX)
middleError = e(middleX)
farError = e(farX)

weightedError = w1*nearError + w2*middleError + w3*farError
```

作用：

把近处、中距离、远处车道误差融合成一个横向控制误差。当前远点权重最大，所以更偏向提前看弯道走势。

7. 子模块四：LKS 方向盘转角命令计算

画布名称：`Steer Command`

源码类：

```
ComputeLksSteerCommand
```

输入：

输入	含义
-----	-----
`weightedError`	三点加权横向误差
`egoV`	自车速度
`controlEnabled`	控制使能

参数：

参数	当前值	含义
-----	-----	-----
`kp`	`0.08`	横向误差比例增益
`steerScale`	`0.6`	转向输出缩放
`ayMax`	`3.0`	最大横向加速度
`wheelBase`	`2.9`	车辆轴距
`frontWheelMaxRad`	`0.5236`	最大前轮转角

内部逻辑：

```
rawSteerNorm = kp * weightedError
speedSquare = egoV * egoV
speedSquareSafe = max(speedSquare, 0.25)

lateralAccelLimitRad = atan(ayMax * wheelBase / speedSquareSafe)
normalizedSteerLimit = lateralAccelLimitRad / frontWheelMaxRad

limitedSteerNorm = clamp(rawSteerNorm,
                        -normalizedSteerLimit,
                        +normalizedSteerLimit)

steerRad = limitedSteerNorm * steerScale * controlEnabled
```

作用：

先用比例控制把横向误差转换为转角，再根据横向加速度上限做速度相关限幅。`controlEnabled=0`时输出转角自动归零。

8. lks_demo 与源码的对应关系

源码文件	画布模块	对应情况
-----	-----	----
`src/LKSAlgorithm.cpp`	`LKS Main Flow`	对应
`src/DetermineLksControlEnabled.cpp`	`Enable Logic`	对应
`src/ComputeLksPreviewDistance.cpp`	`Preview Distance`	对应
`src/ComputeLksWeightedError.cpp`	`Weighted Lane Error`	对应
`src/ComputeLksSteerCommand.cpp`	`Steer Command`	对应
`include/LKSAlgorithm.hpp`	顶层输入/输出/参数/子模块结构	对应

目前从画布结构看，扫描结果基本忠实还原了源码的函数调用关系。

9. 当前可读性问题

9.1 基础块名称重复

扫描器原始结果会把很多基础运算块命名为：

乘法运算
加法运算
变量
常量
读取局部参数

这些名称对代码生成没有问题，但在画布里不容易看出语义。例如：

当前画布名称	实际语义
乘法运算	可能是 $rt * egoV$ 、 $kp * weightedError$ 、 $w1 * nearError$ 等不同含义
加法运算	可能是 $l0 + rt * egoV$ ，也可能是 Horner 多项式累加
变量	可能是 $basePreviewDistance$ 、 $nearError$ 、 $limitedSteerNorm$ 等

9.2 复合块同名 `run`

每个复合块内部函数名都是 `run`，这符合 Pangu 原件结构，但在画布树里容易出现多个 `run`。当前通过英文显示别名区分：

LKS Main Flow
Enable Logic
Preview Distance
Weighted Lane Error
Steer Command

9.3 参数显示分散

参数已经在子模块内部通过 `read-local-param` 读取，但调试人员需要进入子模块才能看到具体参数。

10. 已执行的可读性修改方式

10.1 优先不改算法和连线

当前 `lks\_demo` 已能运行，因此第一阶段只建议做显示层优化：

1. 不改连线。
2. 不改端口名。
3. 不改函数名。
4. 不改参数值。
5. 只改画布中组件的 `alias` / 展示名称。

这样风险最低，基本不会影响代码生成和运行。

10.2 当前复合块显示名

当前已把复合块显示名统一成简单英文，不再使用`00/01/02`编号，也不混用中英文：

源码功能	当前显示名
---	---
`LKSAAlgorithm::run()`	`LKS Main Flow`
`DetermineLksControlEnabled::run()`	`Enable Logic`
`ComputeLksPreviewDistance::run()`	`Preview Distance`
`ComputeLksWeightedError::run()`	`Weighted Lane Error`
`ComputeLksSteerCommand::run()`	`Steer Command`

10.3 当前关键变量显示名

在每个子模块内，已把`变量 | ...`改成有语义的英文显示名：

子模块	变量名	当前显示名
----	-----	-----
预瞄距离	`basePreviewDistance`	`basePreviewDistance`
预瞄距离	`absCurvature`	`absCurvature`
预瞄距离	`curvatureScale`	`curvatureScale`
误差加权	`nearX`	`nearX`
误差加权	`middleX`	`middleX`
误差加权	`farX`	`farX`
误差加权	`nearError`	`nearError`
误差加权	`middleError`	`middleError`
误差加权	`farError`	`farError`
使能判断	`speedReady`	`speedReady`
使能判断	`brakePressed`	`brakePressedFlag`
使能判断	`driverOverride`	`driverOverride`
转角计算	`rawSteerNorm`	`rawSteerNorm`
转角计算	`speedSquareSafe`	`speedSquareSafe`
转角计算	`normalizedSteerLimit`	`normalizedSteerLimit`
转角计算	`limitedSteerNorm`	`limitedSteerNorm`

10.4 当前关键运算块显示名

关键路径上的基础块也已改成英文语义名。对重复出现的`multiply/add/ternary`，优先按公式作用命名，而不是按算子类型命名。

预瞄距离子模块

基础块语义	当前显示名	对应公式
-----	-----	-----
`multiply`	`Speed Preview Term`	$rt * egoV$
`add`	`Base Preview Distance`	$l0 + rt * egoV$
`std::fabs`	`Abs Curvature`	$abs(curvature)$
`greater-equal`	`Curve Threshold Check`	$absCurvature \geq threshold$
`ternary`	`Select Curve Scale`	$rAlpha / 1.0$
`multiply`	`Apply Preview Scale`	$basePreviewDistance * curvatureScale$

三点误差加权子模块

基础块语义	当前显示名	对应公式
-----	-----	-----
`divide`	`Calc middleX`	$\text{previewDistance} / 2$
近点 Horner 链	`Near Horner ...`	$e(\text{nearX})$
中点 Horner 链	`Middle Horner ...`	$e(\text{middleX})$
远点 Horner 链	`Far Horner ...`	$e(\text{farX})$
`w1` 乘法	`Weight near error`	$w1 * \text{nearError}$
`w2` 乘法	`Weight middle error`	$w2 * \text{middleError}$
`w3` 乘法	`Weight far error`	$w3 * \text{farError}$
最后两个加法	`Sum weighted errors`	$w1e1 + w2e2 + w3e3$

使能判断子模块

基础块语义	当前显示名	对应公式
-----	-----	-----
`greater-equal`	`Speed Ready Check`	$\text{egoV} \geq vMin$
`not-equal`	`Brake Pressed Check`	$\text{brakePressed} \neq 0$
`std::fabs`	`Abs Driver Steer`	$abs(\text{driverSteerNorm})$
`greater-equal`	`Driver Override Check`	$abs(\text{driverSteerNorm}) \geq \text{threshold}$
`logic-not`	`Not Braking`	$!brakePressed$
`logic-not`	`No Driver Override`	$!driverOverride$
`logic-and`	`Enable Conditions`	$\text{speedReady} \&\& !brake \&\& !override$

转角命令子模块

基础块语义	当前显示名	对应公式
-----	-----	-----
`multiply`	`Raw Steer Gain`	$kp * \text{weightedError}$
`multiply`	`Speed Square`	$\text{egoV} * \text{egoV}$
`greater-than` + `ternary`	`Safe Speed Square`	$\max(v^2, 0.25)$
`multiply` + `divide` + `atan`	`Lateral Limit atan`	$atan(ayMax * \text{wheelBase} / v^2)$
`divide`	`Normalize Steer Limit`	$\phiLimit / \text{frontWheelMaxRad}$
`fmin/fmax`	`Clamp Upper/Lower Steer`	$\text{clamp}(\text{raw}, -limit, limit)$
最后 `multiply`	`Apply Enable Gate`	$limited * \text{steerScale} * \text{controlEnabled}$

11. 是否需要改源码

如果只想让当前 `lks\_demo` 更好读，可以直接改画布显示名。

如果希望以后重新扫描也能自动更好读，需要改源码注释和局部变量命名：

1. 保留当前类名：`LKSAAlgorithm`、`ComputeLksPreviewDistance` 等。
2. `@cn\_name` 可以继续保留给扫描器和文档使用，但画布最终展示名以扫描后 `alias` 为准。
3. 对重要中间量继续使用语义明确的局部变量名，例如 `basePreviewDistance`、`nearError`、`limitedSteerNorm`。
4. 如果扫描器支持把局部变量名显示成 alias，需要推动工具侧使用变量名而不是统一显示为“变量”。

5. 如果扫描器不支持基础块语义 alias，则只能在扫描后人工或脚本修改 `cbdes.db` 的 alias。

12. 推荐执行顺序

1. 保留当前可运行的 `lks\_demo` 原工程，先备份 `data/cbdes.db`。
2. 只修改显示名，不动连线和端口。
3. 优先改四个子模块内的变量别名。
4. 再改关键运算块别名。
5. 打开 GAASD 检查画布显示是否更清晰。
6. 重新生成代码并运行一次，确认显示名修改没有影响运行。

13. 需要避免的修改

以下修改风险较高，暂时不建议直接做：

1. 改端口名，例如 `egoV`、`weightedError`、`controlEnabled`。
2. 改通道字段名，例如 `lks\_input.ego\_speed\_mps`、`lks\_output.steer\_rad`。
3. 改复合块函数名 `run`。
4. 改参数名，例如 `kp`、`vMin`、`w1/w2/w3`。
5. 手动删基础块或重连线。

这些都会影响代码生成或 Pangu 通道绑定。

14. 补充：LKS 画布详细讲解

`lks\_demo` 的画布看起来比普通公式复杂，是因为它把 LKS 拆成了四个明确的计算阶段：

控制使能判断
-> 远预瞄距离计算
-> 三点预瞄误差加权
-> 方向盘转角命令计算

这四段对应实际 LKS 控制链路中的四个问题：

阶段	回答的问题	输出
---	---	---
`DetermineLksControlEnabled`	当前是否允许 LKS 接管方向盘	`controlEnabled`
`ComputeLksPreviewDistance`	应该看多远的车道线	`previewDistance`
`ComputeLksWeightedError`	车辆相对未来车道中心偏差有多大	`weightedError`
`ComputeLksSteerCommand`	偏差应该转换成多大转向命令	`lksSteerRad`

14.1 控制使能判断：为什么不是一直输出转向

源码逻辑：

```

speedReady = egoV >= vMin
brakePressed = brakePressed != 0
driverOverride = abs(driverSteerNorm) >= driverSteerThreshold

controlEnabled = speedReady AND NOT brakePressed AND NOT driverOverride

```

每个条件的作用：

条件	含义	触发后的影响
<code>egoV >= vMin</code>	车速达到 LKS 工作下限	低速时不启用 LKS
<code>brakePressed == 0</code>	驾驶员没有踩刹车	刹车时退出 LKS
<code>abs(driverSteerNorm) < threshold</code>	驾驶员没有明显主动打方向	人工接管时退出 LKS

这个模块的输出不是转角，而是一个门控信号。后面转角计算完成后，还会乘以 `controlEnabled`：

```
lksSteerRad = steerCommand * controlEnabled
```

所以 `controlEnabled = 0` 时，即使误差计算出了转角，最终输出也会被压成 0。

画布复杂原因：

- `abs(driverSteerNorm)` 会被扫描成绝对值节点。
- 三个条件各自是比较节点。
- `NOT brakePressed`、`NOT driverOverride` 是逻辑非节点。
- 三个条件合成使能需要逻辑与节点。

14.2 远预瞄距离：为什么要根据速度和曲率变化

源码逻辑：

```

basePreviewDistance = l0 + rt * egoV
absCurvature = abs(curvature)
curvatureScale = absCurvature >= curvatureThreshold ? rAlpha : 1.0
previewDistance = basePreviewDistance * curvatureScale

```

物理含义：

中间量	含义
<code>l0</code>	静态基础预瞄距离，保证低速也有最小前视距离
<code>rt * egoV</code>	与速度成正比的动态预瞄距离，速度越高看得越远
<code>curvature</code>	道路曲率，表示道路弯曲程度
<code>curvatureScale</code>	弯道缩短预瞄距离，避免高速弯道看得过远导致转向滞后
<code>previewDistance</code>	远预瞄点位置，后续作为三点误差模型的远点

为什么要这么设计：

- 直道或小曲率道路上，预瞄距离可以长一些，转向更平顺。
- 弯道中如果预瞄距离过长，车辆可能“看过头”，导致转弯响应慢。
- 用 `curvatureScale` 可以在曲率较大时缩短预瞄距离，让控制更积极。

画布复杂原因：

- `l0 + rt * egoV` 会拆成乘法和加法。
- `abs(curvature)` 是绝对值节点。
- 曲率阈值判断是比较节点。
- `rAlpha` 和 `1.0` 的选择是条件选择节点。
- 最终预瞄距离还要乘以缩放系数。

14.3 三点预瞄误差：为什么把 x 带入车道多项式就是误差

输入的车道线多项式是：

$$e(x) = c3 \cdot x^3 + c2 \cdot x^2 + c1 \cdot x + c0$$

这里的坐标系默认是自行车坐标系：

- `x` 表示车辆前方的纵向距离。
- `e(x)` 表示在这个前方距离处，车辆相对目标车道中心线的横向误差。
- `c0~c3` 是每一帧由车道线/道路模型给出的三次多项式系数。

所以把预瞄距离 `x` 代入多项式，得到的不是“道路位置本身”，而是“该预瞄点处的横向偏差”。因为多项式已经是在自行车坐标系下表达的车道中心误差模型。

当前三点为：

```
nearX = nearPreviewDistance
middleX = previewDistance / 2
farX = previewDistance
```

三点误差为：

```
nearError = e(nearX)
middleError = e(middleX)
farError = e(farX)
```

最终加权：

```
weightedError = w1*nearError + w2*middleError + w3*farError
```

三点的作用不同：

点	作用
近点 <code>nearX</code>	反映车辆近处横向偏差，影响快速纠偏
中点 <code>middleX</code>	反映中距离道路趋势，起平衡作用
远点 <code>farX</code>	反映未来道路走势，提前准备弯道

为什么要加权：

- 只看近点容易控制抖动。
- 只看远点容易转向滞后。

- 三点加权能同时兼顾当前纠偏和未来弯道趋势。

画布复杂原因：

- 每个点都要计算一次三次多项式。
- 每次多项式计算会展开成多次乘法和加法。
- 当前源码使用 Horner 形式：

$$e(x) = (((c3 * x) + c2) * x + c1) * x + c0$$

这种写法计算效率高，但扫描到画布后会显示为多级乘法/加法链。

14.4 方向盘转角命令：为什么还有横向加速度限幅

源码逻辑：

```
rawSteerNorm = kp * weightedError
speedSquare = egoV * egoV
speedSquareSafe = max(speedSquare, 0.25)

lateralAccelLimitRad = atan(ayMax * wheelBase / speedSquareSafe)
normalizedSteerLimit = lateralAccelLimitRad / frontWheelMaxRad

limitedSteerNorm = clamp(rawSteerNorm,
    -normalizedSteerLimit,
    +normalizedSteerLimit)

steerRad = limitedSteerNorm * steerScale * controlEnabled
```

各中间量含义：

中间量	含义
rawSteerNorm	根据加权横向误差得到的原始归一化转向命令
speedSquare	车速平方，用于横向加速度约束
speedSquareSafe	防止低速时除以过小数值
lateralAccelLimitRad	由最大横向加速度推算出的前轮转角上限
normalizedSteerLimit	归一化后的转角限制
limitedSteerNorm	限幅后的归一化转向命令
steerRad	最终输出给车辆/桥接层的转角命令，单位 rad

为什么用横向加速度限幅：

车辆横向加速度近似满足：

$$ay \approx v^2 * \tan(\delta) / wheelBase$$

反推转角上限：

$$\deltaLimit = \arctan(ayMax * wheelBase / v^2)$$

速度越高，同样的转角会产生更大的横向加速度，所以高速时转角限制要更严格。这样可以避免 LKS 在高速弯道中给出过大的突变转角。

画布复杂原因：

- ``egoV * egoV`` 是乘法。
- ``max(speedSquare, 0.25)`` 是比较和选择。
- ``atan(ayMax * wheelBase / speedSquareSafe)`` 涉及乘法、除法、反正切。
- ``clamp`` 会拆成上限比较和下限比较。
- 最后还要乘 ``steerScale`` 和 ``controlEnabled``。

14.5 LKS 每一帧是否都要重新计算

需要。LKS 的输入 ``c0~c3``、``curvature``、``egoV``、``brakePressed``、``driverSteerNorm`` 都是运行时输入，每个仿真周期都可能变化：

- 车辆位置变化后，自车坐标系下的车道多项式会变化。
- 速度变化后，预瞄距离会变化。
- 道路曲率变化后，预瞄距离缩放会变化。
- 驾驶员制动或接管会影响使能。

因此，``LKSAAlgorithm`` 每一帧都要重新执行：

```
读取当前输入
-> 计算 controlEnabled
-> 计算 previewDistance
-> 计算 weightedError
-> 计算 lksSteerRad
```

这也是为什么画布中这些模块都应该放在周期执行链路中，而不是只初始化一次。

14.6 汇报时可以这样讲

可以按下面这段话讲：

``lks_demo`` 的核心是基于三次车道多项式的预瞄控制。系统每一帧读取自车速度、车道多项式系数、曲率和驾驶员接管信号。首先判断 LKS 是否允许接管；然后根据车速和道路曲率计算远预瞄距离；再在近、中、远三个预瞄点上代入车道多项式，得到三个横向误差并加权成一个综合误差；最后把综合误差转换成方向盘转角，并通过横向加速度约束和使能信号进行限幅与门控。画布中节点较多，是因为多项式计算、条件判断、限幅和门控都被 GAASD 展开成了基础数据流节点。